Lab 06 – Stack

Task 01:

You're building a **social media app** (such asTikTok or Instagram). Users receive notifications when someone likes their post, comments, or follows them. Notifications are stored in a stack so the latest one appears first. When the user checks notifications, they pop off the stack one by one.

Write a program in C++ to implement the above scenario with following functionalities.

1. Create class Stack.
2. Implement the following functions.

    a. push()
    b. pop()
    c. peek()
    d. isEmpty()
    e. display()

Code:

```
/*Social Media Notofications*/

#include<iostream>
#include<string>
using namespace std;

class NotificationStack{
private:
    static const int capacity = 10;
    string notifications[capacity];
    int top = -1;

public:
    void push(string msg){
        if(top == capacity-1){
            cout << "No more notifications can be added.\n";
            return;
        }
        notifications[++top] = msg;
    }

    void pop(){
        if(top == -1){
            cout << "No notification in the stack.\n";
            return;
```

```
        }
        top--;
    }

    string peek(){
        return notifications[top];
    }

    bool isEmpty(){ return top == -1; }

    void display(){
        if(top == -1){
            cout << "No notifications to display:(\n";
            return;
        }

        for(int i = top; i >= 0; i--){
            cout << i+1 << ". " << notifications[i] << "\n";
        }
        cout << "\n";
    }

};

int main(){

}
```

Task 02:

Consider that you are building a compiler. Implement a C++ program that uses a stack to check whether a given expression has balanced parentheses, square brackets, and curly braces. Provide meaningful error messages for unbalanced cases.

Some Sample Inputs:

- {{[[(())]]}}
- (((())))
- (){}[])
- {[(])}

Code:

```
/*Syntax Checker*/

#include<iostream>
#include<string>
#include<stack>
using namespace std;

bool isOpening(char c){
    return c == '(' || c == '[' || c == '{';
}

char getOpeningBr(char c){
    if(c == ')') return '(';
    if(c == ']') return '[';
    return '{';
}

bool isValid(string s){
    stack<char> st;
    int openCount = 0, closeCount = 0;

    for(int i = 0; i < s.length(); i++){

        // if it is opening bracket push it into stack
        if(isOpening(s[i])){
            st.push(s[i]);
            openCount++;
        }

        // it is a closing bracket
        else{
```

```
            // we got the closing bracket, but there's no opening bracket before
it
            if(st.empty()) return false;

            // get the openeing bracket of it, and compare it with the top of
stack if it is same
            if(getOpeningBr(s[i]) != st.top()){
                return false; // if not, terminating
            }
            else{
                st.pop();
                closeCount++;
            }

        }
    }
    return openCount == closeCount;
}

int main(){

    cout << isValid("");

}
```

Task 03:

In a javelin throw competition, each athlete makes several attempts, and the distance of each throw is recorded. To make reviewing easier, the system stores each throw's distance in a **stack**, ensuring the most recent throw is always on top. For example, after the first attempt, the score of 65 meters is pushed onto the stack. The second attempt of 68 meters is pushed next, and the third attempt of 70 meters is added on top. At this point, the stack holds `[65, 68, 70]` with 70 at the top.

Write a program in C++ to implement the above scenario by considering two players, Maintain two different stacks for both the players, Player A and Player B. Both players have 3 turns for javelin throw. At each time distance is recorded in the stack. Recent recorded distance is shown at the top of the stack. At the time of throw, display the score of both the players and show the scores of the player having the highest throw at each time. Also display the winner of the game.

Code:

```
/*Javelin Throw Score Tracking*/

#include<iostream>
using namespace std;

class Player{
private:
    static const int turns = 3;
    int scores[turns];
    int top = -1;

public:
    void push(int val){
        if(top == turns-1){
            cout << "No more distance can be recorded.\n";
            return;
        }
        scores[++top] = val;
    }
    int seeTop(){ return scores[top]; }
};
int main(){
    Player pa, pb;
    int paScore = 0, pbScore = 0;

    for(int i = 0; i < 3; i++){
        int input = 0;

        cout << "Enter distance for player 1: ";
        cin >> input;
```

```
        pa.push(input);

        cout << "Enter distance for player 2: ";
        cin >> input;
        pb.push(input);

        if(pa.seeTop() > pb.seeTop()){
            cout << "\nIn round " <<i+1<< ", Player A's throw was longest with a
distance of " << pa.seeTop() << "\n\n";
            paScore++;
        }
        else if(pb.seeTop() > pa.seeTop()){
           cout << "\nIn round " <<i+1<< ", Player B's throw was longest with a
distance of " << pb.seeTop() << "\n\n";
           pbScore++;
        }
        else cout << "\nIn round " <<i+1<< " both player have same throwing
distance.\n\n";
    }

    // display winner
    cout << "\n\t\t---RESULT---\n";
    if(paScore > pbScore) cout << "Player A is winner with having " << paScore <<
" throw(s) longeer than player B.\n";
    else if(pbScore > paScore) cout << "Player B is winner with having " <<
pbScore << " throw(s) longeer than player A.\n";
    else cout << "Both player have same score.\n";
}
```

Output:

```
Enter distance for player 1: 4
Enter distance for player 2: 2

In round 1, Player A's throw was longest with a distance of 4

Enter distance for player 1: 5
Enter distance for player 2: 4

In round 2, Player A's throw was longest with a distance of 5

Enter distance for player 1: 5
Enter distance for player 2: 3

In round 3, Player A's throw was longest with a distance of 5


                ---RESULT---
Player A is winner with having 3 throw(s) longeer than player B.
```

Task 04:

A photographer is editing a lawn shoot picture for a clothing brand. The photographer performs several edit operations on the image, and he needs the ability to undo some of the operations if he decides they weren't suitable for the final picture. Each edit will be stored in a stack. Design a C++ program using a stack to perform editing on the image, with a maximum of 10 edits allowed. Implement an undo feature that allows the photographer to revert to a previous edit step. The program should store each edit operation in the stack, and the user should be able to perform edits, view the current image state, and undo edits as needed.

The editing options he might use are:

1. Brightness adjustment (increase or decrease brightness).
2. Cropping (cutting a portion of the image).
3. Applying filters (e.g., grayscale, sepia).

Rotating the image (clockwise or counter-clockwise).

Code:

```
/*Image Editing*/

#include<iostream>
#include<string>
using namespace std;

class EditTimeline{
private:
    static const int maxEdits = 10;
    string arr[maxEdits];
    int top = -1;

public:
    void addEdit(string s){
        if(top == maxEdits-1){
            cout << "No more edits can be done!\n";
            return;
        }
        arr[++top] = s;
        cout << "Edit added!\n";
    }
    void undoEdit(){
        if(top == -1){
            cout << "No editing is done!\n";
            return;
        }
        top--;
        cout << "Edit deleted successfully!\n";
    }
```

```
    void displayEdits(){
        if(top == -1){
            cout << "No editing is done!\n";
            return;
        }
        for(int i = top;i >= 0; i--){
            cout << i+1 << ". " << arr[i] << "\n";
        }
    }
};

int main(){
    EditTimeline editor;
    int choice = 0;
    cout << "\n***Welcome To Image Editor***\n\n";

    do{
        cout << "\n\t---MENU---\n";
        cout << "1. Add Edit.\n";
        cout << "2. Undo Edit.\n";
        cout << "3. See Edit History.\n";
        cout << "4. Exit.\n";
        cout << "Enter choice: ";
        cin >> choice;
        cin.ignore();

        cout << "\n";
        if(choice == 1){
            string msg;
            cout << "Enter edit message: ";
            getline(cin, msg);

            editor.addEdit(msg);
        }
        else if(choice == 2){
            editor.undoEdit();
        }
        else if(choice == 3){
            cout << "\n---Edit Hidtory---\n";
            editor.displayEdits();
        }
        else if(choice == 4){
            cout << "Editor closed.\n";
        }
        else cout << "Invalid choice.\n";
```

```
    }while(choice != 4);
}
```

Output:

```
***Welcome To Image Editor***


        ---MENU---
1. Add Edit.
2. Undo Edit.
3. See Edit History.
4. Exit.
Enter choice: 1

Enter edit message: resize
Edit added!

        ---MENU---
1. Add Edit.
2. Undo Edit.
3. See Edit History.
4. Exit.
Enter choice: 1

Enter edit message: brightness adjusted
Edit added!

        ---MENU---
1. Add Edit.
2. Undo Edit.
3. See Edit History.
4. Exit.
Enter choice: 3
```

```
Enter choice: 3


---Edit Hidtory---
2. brightness adjusted
1. resize

        ---MENU---
1. Add Edit.
2. Undo Edit.
3. See Edit History.
4. Exit.
Enter choice: 4

Editor closed.
```